

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №3

Выполнил:

Грищук Елизавета Дмитриевна
БР 2.1

Проверил:

Добряков Д. И.

Санкт-Петербург

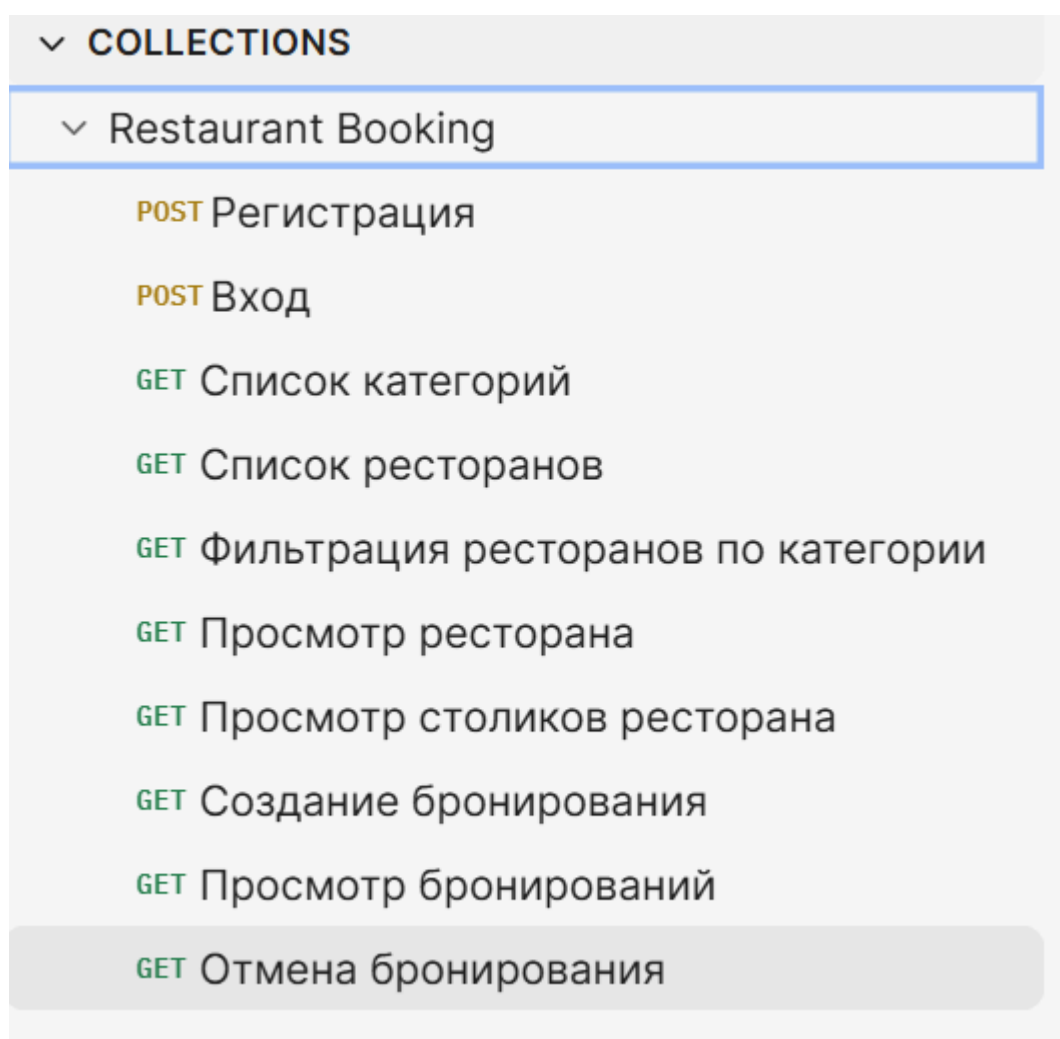
2026 г.

Задача

Реализовать тестирование REST API средствами Postman. Написать тесты внутри Postman, реализовав один комплексный сценарий по основному процессу приложения бронирования столиков в ресторанах.

Ход работы

В Postman была создана коллекция Restaurant Booking. Для передачи данных между запросами настроены переменные коллекции: base_url, token, restaurant_id, table_id, booking_id.



Реализован следующий комплексный сценарий:

1. Регистрация пользователя — POST /auth/register. Создаётся новый пользователь. Тест проверяет статус 201 и наличие полей id и email в ответе.

Restaurant Booking > Регистрация

POST {{base_url}} /auth/register

Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

Pre-req

Post-res

```
1 pm.test("Статус 201", () => {
2   pm.response.to.have.status(201);
3 });
4
5 pm.test("Есть id и email", () => {
6   const json = pm.response.json();
7   pm.expect(json).to.have.property("id");
8   pm.expect(json).to.have.property("email");
9 });
```

Body

201 Created • 149 ms • 300 B • Save Response

{ JSON Preview Visualize

```
1 {
2   "id": 1,
3   "name": "Елизавета",
4   "email": "liza@test.com"
5 }
```

2. Авторизация — POST /auth/login. Пользователь входит в систему. Тест проверяет статус 200, наличие токена и сохраняет его в переменную token.

Restaurant Booking > Вход

Save Share

POST {{base_url}}/auth/login Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

Pre-req
Post-res

```
1 pm.test("Статус 200", () => {  
2   pm.response.to.have.status(200);  
3 });  
4  
5 pm.test("Есть токен", () => {  
6   const json = pm.response.json();  
7   pm.expect(json).to.have.property("token");  
8   pm.collectionVariables.set("token", json.token);  
9 });
```

Body 200 OK • 94 ms • 391 B • Save Response

{ } JSON Preview Visualize

```
1 {  
2   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.  
3     eyJ1c2VySWQiOiJEsIm1hdCI6MTc4MTYwMjg4NywiZmxhZjoxNzgxNjg5Mjg3fQ.  
4     Z7c3RlI4kpuEQX7HIWlsJ67X2kGwQtKlSpY6pnm3BLM"
```

3. Список категорий — GET /categories. Получение всех категорий кухни. Тест проверяет статус 200 и что ответ является массивом.

The screenshot shows a REST client interface with the following details:

- URL:** `{{base_url}}/categories`
- Method:** GET
- Scripts:**

```
1 pm.test("Статус 200", () => {
2   pm.response.to.have.status(200);
3 });
4
5 pm.test("Ответ是 массив", () => {
6   const json = pm.response.json();
7   pm.expect(json).to.be.an("array");
8 });
```
- Response:** 200 OK • 31 ms • 315 B
- Body (JSON):**

```
1 [
2   {
3     "id": 4,
4     "name": "Italian"
5   },
6   {
7     "id": 5,
8     "name": "Japanese"
9   },
10  {
11    "id": 6,
12    "name": "Russian"
13  }
14 ]
```

4. Список ресторанов — GET /restaurants. Получение всех ресторанов. Тест проверяет статус 200, непустой массив и сохраняет restaurant_id первого ресторана в переменную.

Restaurant Booking > Список ресторанов

GET {{base_url}}/restaurants

Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Pre-req

Post-res

```
1 pm.test("Статус 200", () => {
2   pm.response.to.have.status(200);
3 });
4
5 pm.test("Ответ - массив", () => {
6   const json = pm.response.json();
7   pm.expect(json).to.be.an("array");
8   pm.expect(json.length).to.be.above(0);
9   pm.collectionVariables.set("restaurant_id", json[0].id);
10 });
```

Body 200 OK • 32 ms • 589 B • Save Response

JSON Preview Visualize

```
1 [
2   {
3     "id": 5,
4     "categoryId": 4,
5     "name": "La Bella Italia",
6     "address": "Nevskaya st. 10",
7     "priceRange": 2,
8     "description": "Italian restaurant",
9     "photoUrl": null,
10    "category": {
11      "id": 4,
12      "name": "Italian"
13    }
14  },
15  {
16    "id": 6,
```

5. Фильтрация ресторанов по категории — GET /restaurants?category_id=4. Тест проверяет что все возвращённые рестораны принадлежат указанной категории.

Restaurant Booking > Фильтрация ресторанов по категории

GET {{base_url}}/restaurants?category_id=4

Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Pre-req

Post-res

```
1 pm.test("Статус 200", () => {
2   pm.response.to.have.status(200);
3 });
4
5 pm.test("Только рестораны нужной категории", () => {
6   const json = pm.response.json();
7   pm.expect(json).to.be.an("array");
8   json.forEach(r => {
9     pm.expect(r.categoryId).to.equal(4);
10  });
11 });
```

Body

200 OK • 4 ms • 417 B • Save Response

JSON Preview Visualize

```
1 [
2   {
3     "id": 5,
4     "categoryId": 4,
5     "name": "La Bella Italia",
6     "address": "Nevskaya st. 10",
7     "priceRange": 2,
8     "description": "Italian restaurant",
9     "photoUrl": null,
10    "category": {
11      "id": 4,
12      "name": "Italian"
13    }
14  }
15 ]
```

6. Просмотр конкретного ресторана — GET /restaurants/{{restaurant_id}}. Тест проверяет статус 200 и наличие полей id и name.

Restaurant Booking > Просмотр ресторана

GET {{base_url}}/restaurants/{{restaurant_id}}

Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Pre-req

Post-res

```
1 pm.test("Статус 200", () => {
2   pm.response.to.have.status(200);
3 });
4
5 pm.test("Правильный ресторан", () => {
6   const json = pm.response.json();
7   pm.expect(json).to.have.property("id");
8   pm.expect(json).to.have.property("name");
9 });
```

Body

200 OK • 37 ms • 415 B • Save Response

{ JSON Preview Visualize

```
1 {
2   "id": 5,
3   "categoryId": 4,
4   "name": "La Bella Italia",
5   "address": "Nevskaya st. 10",
6   "priceRange": 2,
7   "description": "Italian restaurant",
8   "photoUrl": null,
9   "category": {
10    "id": 4,
11    "name": "Italian"
12  }
13 }
```

7. Просмотр столиков ресторана — GET /restaurants/{{restaurant_id}}/tables. Тест проверяет непустой массив столиков и сохраняет table_id в переменную.

Restaurant Booking > Просмотр столиков ресторана

GET {{base_url}}/restaurants/{{restaurant_id}}/tables

Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Pre-req

Post-res

```
1 pm.test("Статус 200", () => {
2   pm.response.to.have.status(200);
3 });
4
5 pm.test("Есть столики", () => {
6   const json = pm.response.json();
7   pm.expect(json).to.be.an("array");
8   pm.expect(json.length).to.be.above(0);
9   pm.collectionVariables.set("table_id", json[0].id);
10  });
```

Body

200 OK • 142 ms • 357 B • Save Response

{ JSON Preview Visualize

```
1 [
2   {
3     "id": 16,
4     "restaurantId": 5,
5     "capacity": 2
6   },
7   {
8     "id": 17,
9     "restaurantId": 5,
10    "capacity": 4
11  },
12  {
13    "id": 18,
14    "restaurantId": 5,
15    "capacity": 6
16  }
17 ]
```

8. Создание бронирования — POST /bookings. Выполняется с JWT-токеном в заголовке Authorization. Тест проверяет статус 201, наличие id и статус pending, сохраняет booking_id в переменную.

HTTP Restaurant Booking > Создание бронирования

POST {{base_url}}/bookings

Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "table_id": {{table_id}},
3   "booking_time": "2026-03-20T19:00:00.000Z",
4   "guests_count": 2
5 }
```

Body 201 Created • 146 ms • 349 B • Save Response

JSON Preview Visualize

```
1 {
2   "userId": 1,
3   "tableId": 16,
4   "bookingTime": "2026-03-20T19:00:00.000Z",
5   "guestsCount": 2,
6   "status": "pending",
7   "id": 1
8 }
```

9. Просмотр бронирований — GET /bookings. Выполняется с токеном. Тест проверяет статус 200 и непустой массив.

Restaurant Booking > Просмотр бронирований

GET {{base_url}}/bookings

Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Pre-req

Post-res

```
1 pm.test("Статус 200", () => {
2   pm.response.to.have.status(200);
3 });
4
5 pm.test("Есть бронирования", () => {
6   const json = pm.response.json();
7   pm.expect(json).to.be.an("array");
8   pm.expect(json.length).to.be.above(0);
9 });
```

Body 200 OK • 40 ms • 551 B • Save Response

JSON Preview Visualize

```
1 {
2   "id": 1,
3   "userId": 1,
4   "tableId": 16,
5   "bookingTime": "2026-03-20T19:00:00.000Z",
6   "guestsCount": 2,
7   "status": "pending",
8   "table": {
9     "id": 16,
10    "restaurantId": 5,
11    "capacity": 2,
12    "restaurant": {
13      "id": 5,
14      "categoryId": 4,
15      "name": "La Bella Italia",
16      "address": "Nevskaya st. 10",
17      "priceRange": 2,
18      "description": "Italian restaurant",
19      "photoUrl": null
20    }
21  }
```

10. Отмена бронирования — PATCH /bookings/{{booking_id}}/cancel. Выполняется с токеном. Тест проверяет статус 200 и статус cancelled в ответе.

Restaurant Booking > Отмена бронирования

PATCH {{base_url}}/bookings/{{booking_id}}/cancel Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

Pre-req
Post-res

```
1 pm.test("Статус 200", () => {  
2   pm.response.to.have.status(200);  
3 });  
4  
5 pm.test("Статус отменён", () => {  
6   const json = pm.response.json();  
7   pm.expect(json.status).to.equal("cancelled");  
8 });
```

Body 200 OK • 38 ms • 346 B • Save Response

{ JSON Preview Visualize

```
1 {  
2   "id": 1,  
3   "userId": 1,  
4   "tableId": 16,  
5   "bookingTime": "2026-03-20T19:00:00.000Z",  
6   "guestsCount": 2,  
7   "status": "cancelled"  
8 }
```

Вывод

В ходе выполнения работы был реализован комплексный сценарий тестирования REST API приложения бронирования столиков в ресторанах средствами Postman. Сценарий включает 10 последовательных запросов, охватывающих основной процесс: от регистрации пользователя до отмены бронирования. Для каждого запроса написаны автоматические тесты, проверяющие корректность HTTP-статусов и структуры ответов. Передача данных между запросами реализована через переменные коллекции.